TASK 12 – Re-evaluate and Adapt the Design

A. Impact analysis

The analyser now supplies fat and SNF to two decimal places, the April–June seasonal base rate is Rs 32.00, the fat step value is Rs 0.45, and farmers supplying more than 500 litres in ten days receive Rs 0.50 per litre bonus. The original constants are no longer sufficient because rates depend on season. The step calculation must support hundredth-place readings while retaining the specified 0.1 percentage-point step. Use Decimal throughout so that two-decimal readings do not introduce binary floating-point error. The bonus applies only when the ten-day quantity is strictly greater than 500 litres.

B. Evaluation with examples

For April–June, fat 3.57 and SNF 8.54 differ from the reference values by 0.07 and 0.04 percentage points. Dividing by the 0.1 percentage-point step gives 0.7 fat steps and 0.4 SNF steps; rounding to the nearest whole step gives +1 fat step and 0 SNF steps. If the seasonal SNF adjustment remains Rs 0.30 per step because the changed requirement only revises the fat step value, the rate is Rs 32.00 + (1 × Rs 0.45) + (0 × Rs 0.30) = Rs 32.45 per litre. The brief does not state a seasonal SNF-step change, so the original Rs 0.30 is retained as an explicit assumption. A delivery of 10 litres at this rate is Rs 324.50. At exactly 500 litres no bonus applies; at 500.01 litres the bonus applies to all eligible litres only if the union’s policy defines it that way. The wording says farmers supplying more than 500 litres receive a bonus per litre, so this implementation applies the bonus to all ten-day litres once the threshold is exceeded.

C. Adapted implementation

```
from decimal import Decimal, ROUND_HALF_UP

STEP = Decimal("0.1")
REFERENCE_FAT = Decimal("3.5")
REFERENCE_SNF = Decimal("8.5")
BONUS_THRESHOLD = Decimal("500")
BONUS_PER_LITRE = Decimal("0.50")
CENT = Decimal("0.01")

RATE_CARDS = {
"regular": {"base": Decimal("30.00"), "fat_step": Decimal("0.40"), "snf_step": Decimal("0.30")},
"april_june": {"base": Decimal("32.00"), "fat_step": Decimal("0.45"), "snf_step": Decimal("0.30")},
}

def rounded_steps(reading, reference):
return ((reading - reference) / STEP).quantize(Decimal("1"), rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf, season="regular"):
card = RATE_CARDS[season]
fat_steps = rounded_steps(fat, REFERENCE_FAT)
snf_steps = rounded_steps(snf, REFERENCE_SNF)
return (card["base"] + fat_steps * card["fat_step"] + snf_steps * card["snf_step"]).quantize(CENT,
rounding=ROUND_HALF_UP)

def ten_day_payment(total_litres, fat, snf, season="regular"):
rate = rate_per_litre(fat, snf, season)
base_payment = total_litres * rate
bonus = total_litres * BONUS_PER_LITRE if total_litres > BONUS_THRESHOLD else Decimal("0.00")
return (base_payment + bonus).quantize(CENT, rounding=ROUND_HALF_UP)

if __name__ == "__main__":
fat, snf = Decimal("3.57"), Decimal("8.54")
for season in ("regular", "april_june"):
print(season, "rate=", rate_per_litre(fat, snf, season))
print("500 L total=", ten_day_payment(Decimal("500"), fat, snf, "april_june"))
```

```
print("501 L total=", ten_day_payment(Decimal("501"), fat, snf, "april_june"))
```

D. Evidence-based defence and trade-offs

The modified design is preferred over a full redesign. The core functions, validation and module structure remain useful; the rate card becomes season-dependent and a separate ten-day bonus calculation is added. Test both rate cards using the same readings, test 500 litres and a value above 500 litres, and confirm that the bonus is absent at the threshold and present above it. The design is easier to maintain because the seasonal rates are data rather than duplicated expressions. The trade-offs are configuration complexity and the need to define effective dates, the bonus scope (all litres or only litres above 500), the seasonal SNF adjustment and how ten-day volume is aggregated across shifts. These unresolved business rules should be confirmed with the union before production deployment.